

1 Formalized TypeSpec and Elixir Type

Annotations	A	$::=$	$\overline{\text{@spec } E} \mid \varepsilon$
Expressions	E	$::=$	$x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T	$::=$	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\bar{T}\} \mid \% \{\bar{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \llcorner \mid \llcorner :: n \llcorner \mid \llcorner :: _ * n \llcorner \mid \llcorner :: n, _ :: _ * n \llcorner$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	$::=$	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names)

Figure 1: TypeSpec

Annotations	A	$::=$	$\$S \mid \$S \text{ when } \overline{\alpha : \bar{T}}$
Signatures	S	$::=$	$\bar{T} \rightarrow T \mid S \text{ and } S$
Types	T	$::=$	$\alpha \mid n..n \mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$ $\mid K \mid \text{term}() \mid \text{none}() \mid \text{dynamic}()$ $\mid \{\bar{T}\} \mid \{\bar{T}, \dots\} \mid \% \{\bar{F}\} \mid \% \{\dots, \bar{F}\}$ $\mid (\bar{T} \rightarrow T) \mid \text{empty_list}() \mid \text{non_empty_list}(T, T)$
Key types	K	$::=$	$\text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}() \mid \text{binary}()$ $\mid \text{tuple}() \mid \text{open_map}() \mid \text{fun}()$ $\mid \text{list}()$
Fields	F	$::=$	$:k \Rightarrow T \mid K \Rightarrow \text{if_set}(T)$

(where x ranges over expression variables, α ranges over type variables, $n..n$ ranges over integers while invisible to the users, and k ranges over atom names)

Figure 2: Elixir Type

2 TypeSpec Built-in Types and Syntactic Sugars

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:__struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpec Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k: T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpec Syntactic Sugars

3 Elixir Type Built-in Types and Syntactic Sugars

Built-in types	Defined as
non_empty_list(T)	non_empty_list(T, empty_list())
list(T)	empty_list() or non_empty_list(T)
list()	list(term())
improper_list()	non_empty_list(term(), not list())

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
{...}	tuple()
%{...}	open_map()
k: T	:k => T
K=>not_set()	K=>if_set(none())

Table 4: Elixir Type Syntactic Sugars

4 Translation from TypeSpec to Elixir Type

$\llbracket \cdot \rrbracket$: TypeSpec $\rightarrow$ Elixir type

Figure 3: Translation

4.1 Translation in Annotations

$$\begin{aligned}
\llbracket @\text{spec } E \rrbracket &= \$ \llbracket E \rrbracket \\
\llbracket @\text{spec } \bar{E}, @\text{spec } E' \rrbracket &= \$ \llbracket \bar{E}, E' \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \text{ when } \overline{\alpha : T_3} \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\
\\
\llbracket \bar{E}, x(\bar{T}_1) :: T_2 \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2' \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2' \rrbracket & \text{otherwise} \end{cases} \\
\\
\llbracket \bar{E}, x(\bar{T}_1) :: T_2 \text{ when } \overline{\alpha' : T_3'} \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2' \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket}, \overline{\alpha' : \llbracket T_3' \rrbracket} & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ & \text{and } \overline{\alpha} \cap \overline{\alpha'} = \emptyset \\ S \text{ and } \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2' \rrbracket[\alpha' := \beta'] \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket}, \overline{\beta' : \llbracket T_3' \rrbracket} & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ & \text{and } \overline{\alpha} \cap \overline{\alpha'} \neq \emptyset \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2' \rrbracket \text{ when } \overline{\alpha' : \llbracket T_3' \rrbracket} & \text{otherwise} \end{cases} \\
\\
\llbracket \epsilon \rrbracket &=
\end{aligned}$$

Figure 4: Translation in Annotations

4.2 Translation in Types

$\llbracket \alpha \rrbracket = \alpha$
 $\llbracket T_1 | T_2 \rrbracket = \llbracket T_1 \rrbracket \text{ or } \llbracket T_2 \rrbracket$
 $\llbracket \text{any}() \rrbracket = \text{term}()$
 $\llbracket \text{none}() \rrbracket = \text{none}()$
 $\llbracket \text{atom}() \rrbracket = \text{atom}()$
 $\llbracket :k \rrbracket = :k$
 $\llbracket \text{pid}() \rrbracket = \text{pid}()$
 $\llbracket \text{port}() \rrbracket = \text{port}()$
 $\llbracket \text{reference}() \rrbracket = \text{reference}()$
 $\llbracket \text{float}() \rrbracket = \text{float}()$
 $\llbracket \text{integer}() \rrbracket = \text{integer}()$
 $\llbracket \text{neg_integer} \rrbracket = -\text{int} -- -1$
 $\llbracket \text{non_neg_integer} \rrbracket = 0 -- \text{inf}$
 $\llbracket \text{pos_integer} \rrbracket = 1 -- \text{inf}$
 $\llbracket \text{tuple}() \rrbracket = \text{tuple}()$
 $\llbracket \{\bar{T}\} \rrbracket = \{\llbracket \bar{T} \rrbracket\}$
 $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \bigoplus_{i=1..n} \llbracket \% \{F_i\} \rrbracket$
 $\llbracket [] \rrbracket = \text{empty_list}()$
 $\llbracket \text{nonempty_maybe_improper_list}(T_1, T_2) \rrbracket = \text{non_empty_list}(\llbracket T_1 \rrbracket, \llbracket T_2 \rrbracket)$
 $\llbracket \langle \rangle \rrbracket = \text{undefined}$
 $\llbracket \langle_ :: n \rangle \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$
 $\llbracket \langle_ ::_ * n \rangle \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$
 $\llbracket \langle_ :: n, _ ::_ * n \rangle \rrbracket = \begin{cases} \text{binary}() & \text{if } n = 8 \\ \text{undefined} & \text{otherwise} \end{cases}$
 $\llbracket (\bar{T} \rightarrow T) \rrbracket = (\llbracket \bar{T} \rrbracket \rightarrow \llbracket T \rrbracket)$
 $\llbracket (\dots \rightarrow T) \rrbracket = \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{undefined} & \text{otherwise} \end{cases}$
 $\llbracket n \rrbracket = n -- n$
 $\llbracket n..n' \rrbracket = n -- n'$
 (where $\oplus$ merges field types and $--$ is invisible to the users)

Figure 5: Translation in Types

4.3 Translation in Fields

$$\llbracket \% \{ \text{required}(T) \Rightarrow T' \} \rrbracket = \begin{cases} \% \{ :k \Rightarrow \llbracket T' \rrbracket \} & \text{if } \llbracket T \rrbracket \approx :k \\ \% \{ K \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \} \text{ and not } \% \{ \} & \text{if } \llbracket T \rrbracket \approx K \text{ and } \llbracket T \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\llbracket \% \{ \text{optional}(T) \Rightarrow T' \} \rrbracket = \begin{cases} \% \{ K \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \} & \text{if } \llbracket T \rrbracket \approx K \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\% \{ :k \Rightarrow \llbracket T' \rrbracket \} \equiv \langle \llbracket T' \rrbracket^{:k}; [\emptyset]; [\text{term}()] \rangle$$

$$\% \{ K \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \} \text{ and not } \% \{ \} \equiv \langle [\emptyset]; [\llbracket T' \rrbracket^K]; [\llbracket T' \rrbracket^K, \text{term}()] \rangle$$

$$\% \{ K \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \} \equiv \langle [\emptyset]; [\llbracket T' \rrbracket^K]; [\text{term}()] \rangle$$

$$\llbracket \% \{ F_1 \} \rrbracket \oplus \llbracket \% \{ F_2 \} \rrbracket = \begin{cases} \langle \llbracket T'_1 \rrbracket^{:k}; [\llbracket T'_2 \rrbracket^{K_2} \setminus \llbracket T'_2 \rrbracket^{:k}]; [\text{term}()] \rangle & \text{if } \llbracket T_1 \rrbracket \approx :k \text{ and } \llbracket T_2 \rrbracket \approx \text{atom}() \\ \langle [\emptyset]; [\llbracket T'_2 \rrbracket^{K_2}]; [\llbracket T'_2 \rrbracket^{K_2}, \text{term}()] \rangle & \text{if } \llbracket T_1 \rrbracket \approx \text{atom}() \text{ and } \llbracket T_2 \rrbracket \approx :k \\ \llbracket \% \{ F_1 \} \rrbracket & \text{if } K_1 \approx K_2 \\ \llbracket \% \{ F_1 \} \rrbracket \cup \llbracket \% \{ F_2 \} \rrbracket & \text{if } K_1 \not\approx K_2 \\ \text{undefined} & \text{otherwise} \end{cases}$$

Old :

$$\bigoplus_{i=1..n} \llbracket \% \{ F_i \} \rrbracket = \langle [\bigvee_{\alpha} (\llbracket T'_{\alpha} \rrbracket^{:k_{\alpha}})]; [\bigcup_{\beta} \llbracket T'_{\beta} \rrbracket^{K_{\beta}}]; [\bigcup_{\beta} \llbracket T'_{\beta} \rrbracket^{K_{\beta}} \vee \text{term}()] \rangle \quad \text{if } :k_{\alpha} \not\leq K_{\beta} \text{ and } K_{\beta_j} \not\approx K_{\beta_h}$$

$$\llbracket \% \{ \text{required}(T) \Rightarrow T' \} \rrbracket = \begin{cases} \langle \llbracket K \rrbracket; [\text{if_set}(\llbracket T' \rrbracket)]; [\emptyset] \rangle \setminus \langle [\perp]; [\perp]; [\emptyset] \rangle & \text{if } \llbracket T \rrbracket \approx K \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\llbracket \% \{ \text{optional}(T) \Rightarrow T' \} \rrbracket = \begin{cases} \langle \llbracket K \rrbracket; [\text{if_set}(\llbracket T' \rrbracket)]; [\emptyset] \rangle & \text{if } \llbracket T \rrbracket \approx K \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\begin{aligned} & \bigoplus_{i=1..n} \llbracket \% \{ \text{required}(T_i) \Rightarrow T'_i \} \rrbracket \\ &= \langle [\bigwedge_i K_i]; [\bigwedge_i \text{if_set}(\llbracket T'_i \rrbracket)]; [\bigcup_i S_i] \rangle \end{aligned}$$

$$\begin{aligned} & \bigoplus_{i=1..n} \llbracket \% \{ \text{required}(T_i) \Rightarrow T'_i \} \rrbracket \\ &= \langle [\bigwedge_i K_i]; [\bigwedge_i \text{if_set}(\llbracket T'_i \rrbracket)]; [\bigcup_i S_i] \rangle \bigoplus_{i=1..n} \llbracket \% \{ \text{required}(T_i) \Rightarrow T'_i \} \rrbracket \oplus \langle [\perp]; [\perp]; [\emptyset] \rangle \\ &= \langle [\bigwedge_i K_i]; [\bigwedge_i \text{if_set}(\llbracket T'_i \rrbracket)]; [\bigcup_i S_i] \rangle \setminus \langle [\perp]; [\perp]; [\emptyset] \rangle \wedge \langle [\perp]; [\perp]; [\emptyset] \rangle \end{aligned}$$

$$\begin{aligned} & \bigoplus \llbracket \% \{ \dots, \text{optional}(T_1) \Rightarrow T'_1 \} \rrbracket \oplus \llbracket \% \{ \text{optional}(T_2) \Rightarrow T'_2 \} \rrbracket \\ &= \bigoplus \llbracket \% \{ \dots, \text{optional}(T_1) \Rightarrow T'_1 \} \rrbracket \cup \llbracket \% \{ \text{optional}(T_2) \Rightarrow T'_2 \} \rrbracket \end{aligned}$$

$$\llbracket \% \{ \text{required}(T) \Rightarrow T' \} \rrbracket = \begin{cases} \% \{ :k \Rightarrow \llbracket T' \rrbracket \} & \text{if } \llbracket T \rrbracket \approx :k \\ \% \{ K \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \} \text{ and not } \% \{ \} & \text{if } \llbracket T \rrbracket \approx K \text{ and } \llbracket T \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\llbracket \% \{ \text{optional}(T) \Rightarrow T' \} \rrbracket = \begin{cases} \% \{ K \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \} & \text{if } \llbracket T \rrbracket \approx K \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\begin{aligned} & \llbracket \% \{ \text{required}(T_1) \Rightarrow T'_1 \} \rrbracket \oplus \llbracket \% \{ \text{required}(T_2) \Rightarrow T'_2 \} \rrbracket \\ & \quad \left(\% \{ :k_1 \Rightarrow \llbracket T'_1 \rrbracket, :k_2 \Rightarrow \llbracket T'_2 \rrbracket \} \right) \quad \text{if } \llbracket T_1 \rrbracket \approx :k_1 \text{ and } \llbracket T_2 \rrbracket \approx :k_2 \end{aligned}$$